

```
<source src="movie.ogv" type='video/ogg; codecs="theora,
vorbis"' />
</video>
```

The full level of integration of the video tag as a native browser element deprecates issues that might arise with third party embedded players such as drop down menus and iframes overlaying on the player or weird layout behavior when the other HTML elements surrounding the video were dynamically resized. Since the video is not treated as an embedded foreign object, there are some other benefits that affect the user experience. For instance, even if the focus is on the player itself actions like page scrolling or keyboard key strokes will be completely functional.

b. VIDEO + JS

The video tag comes with a set of attributes and methods which let you have a fine control of your video from your JS code. As any other HTML element you can attach common events to the video tag such as drag events, mouse events, focus events, etc. Since loading a video resource can have many caveats there are many events to have a fine control of the loading process, both at network level (loadstart, progress, suspend, abort, error, emptied, stalled) and at buffering level (loadedmetadata, loadeddata, waiting, playing, canplay, canplaythrough).

At its simplest level you can attach the canplay event to start doing stuff with the video.

```
• video.addEventListener('canplay', function(e) {
  this.volume = 0.4;
  this.currentTime = 10;
  this.play();
}, false);
```

c. VIDEO + CSS

The video tag can also be styled using traditional CSS (e.g. border, opacity, etc) since it is a first-class citizen in the DOM. But the cool thing is that you can even style it with the latest CSS3 properties like reflections, masks, gradients, transforms, transitions and animations.

d. VIDEO + CANVAS

Canvas holds loads of possibilities when used in conjunction with the video tag. You can make use of the following features –

1. Import images - The drawImage method lets you import images from three